

Phase 3: Project Design

3.1 System Architecture

Three-tier design: **Streamlit frontend** → **FastAPI backend** → **Gemini AI core**, plus formatting modules for export.

```
flowchart LR
    U["User Interface (Streamlit)"] -->| "JSON: document_t" | B
    B --> G["Document Generator (gemini_generator.py)"]
    G -->| prompt | A["Gemini API (Google)"]
    A -->| generated text | G
    G --> B
    B -->| "JSON: document" | U
    U --> F["Formatting Modules (sanitize, HTML preview, ...)"]
    F --> O["Output files (.txt, .docx, .pdf)"]
```

Component	Responsibility
Frontend (Streamlit)	Collect inputs, show preview, allow editing, provide download buttons
Backend (FastAPI)	Validate request (Pydantic), route to generator, return document text
AI Core	Build structured prompt, call Gemini generate_content, return text
Formatting Modules	sanitize_text, format_html_preview, format_docx, format_pdf

3.2 Sequence Diagram

```
sequenceDiagram
    participant User
    participant UI as Streamlit UI
    participant API as FastAPI /generate
    participant Gen as GeminiDocumentGenerator
    participant LLM as Gemini API
    User->>UI: Fill form, click Generate Document
    UI->>API: POST /generate (JSON)
    API->>Gen: generate_document(type, parties, terms, da
    Gen->>LLM: generate_content(prompt)
    LLM-->>Gen: response.text
    Gen-->>API: document text
    API-->>UI: {"document": "..."}
    UI->>UI: sanitize_text, render preview
    User->>UI: (optional) Edit document
    User->>UI: Click Download TXT / DOCX / PDF
    UI-->>User: File download
```

3.3 Folder Structure

```
LEGALEASE/
├── ai_core/
│   ├── gemini_generator.py    # GeminiDocumentGenerator
│   └── generator.py          # sanitize_text, format_h
├── frontend/
│   └── app.py                 # Streamlit UI
├── Image/
│   ├── Logo.png              # logo for DOCX/PDF
│   └── inverseLogo.png       # logo for dark-themed we
├── legalEaseAPI/
│   ├── main.py               # FastAPI app, "/" route,
│   └── routes.py             # DocumentRequest model,
├── docs/                     # phase-wise documentatio
├── config.py                 # paths, backend URL, mod
├── requirements.txt
└── run.sh                   # starts backend and fron
```

```
| .env
| .gitignore
```

```
# GEMINI_API_KEY (NOT com
```

3.4 API Design

GET /

Health check.

```
{ "message": "Welcome to LegalEase AI Legal Document Gene
```

POST /generate

Request body (DocumentRequest):

```
{
  "document_type": "Freelance Work Contract",
  "parties": "Jane Doe (Service Provider), TechNova Inc.",
  "terms": "Work must be delivered by May 15, 2025; Payme",
  "dates": "April 15, 2025"
}
```

Success response (200):

```
{ "document": "### Freelance Work Contract\n\nAgreement ma
```

Error responses: 422 invalid/missing fields (Pydantic), 500 Gemini/API failure (returned with a readable message).

3.5 Prompt Design

Generate a comprehensive legal document titled '{document
Involved parties: {parties}
Effective Date: {dates}
Terms and conditions: {terms}
Ensure formal legal structure with multiple sections and

Recommended prompt guardrails (added during development):

- Use numbered section headings and plain text (no markdown symbols)
- Include parties, recitals, definitions, term and termination, governing law, severability, signature block
- Use bracketed placeholders (e.g. [Address]) only for details the user did not supply
- Do not invent facts not provided

3.6 UI Design (Wireframe)

[LegalEase logo]
AI Legal Document Generator

Document Type (Ex: Agreement, Contract, NDA)
[_____]

Parties Involved
[_____]

Terms & Conditions (Use semicolons for bullets)
[_____]

Effective Date
[_____]

[Generate Document]

(after generation)
[Document Generated Successfully!]

Styled scrollable preview (dark card)

```
| [ Click to Edit Document ] |
| [ Download as .TXT ] [ .DOCX ] [ .PDF ] |
+-----+-----+-----+
```

Design choices: centered layout, three-column trick ([1, 2, 1]) to center the logo, dark theme preview card, emoji-labelled download buttons.

3.7 Module Design

Module	Function / Class
gemini_generator.py	GeminiDocumentGenerator.generate_docu
generator.py	sanitize_text(text)
generator.py	format_html_preview(text)
generator.py	format_docx(text, doc_type, terms)
generator.py	format_pdf(text, doc_type)
routes.py	generate_legal_document()

3.8 Document Layout Design

Format	Layout
DOCX	Logo (centered, top), title, body paragraphs in Times New Roman, bold section headings, terms table auto-built from semicolon-separated terms, footer
PDF	Header with centered logo and title on every page, bold headings, bullet-style terms, footer "LegalEase Inc. contact@legalease.com All Rights Reserved."
TXT	Plain generated (or edited) text

3.9 Security Design

- `GEMINI_API_KEY` loaded from `.env` using `python-dotenv`; `.env` listed in `.gitignore`
- Pydantic validation on all incoming data
- AI output HTML-escaped before rendering in Streamlit (`unsafe_allow_html`)
- No user documents stored on the server